RV College of Engineering
Department of Computer Science and Engineering

College	**RV COLLEGE OF ENGINEERING**		
Department	**COMPUTER SCIENCE AND ENGINEERING**		
PROGRAM: BE-CSE	Cloud Computing- Elective Project Based Learning	**Course Code**	IS355TBD
Student Name1	Dev Sisodia	**USN**	1RV24CS071
Student Name2	Ishaan Snehal Parikh	**USN**	1RV24CS105
Student Name3	K Vikas	**USN**	1RV24CS116
Student Name4	Kartik Shekhar	**USN**	1RV24CS124
Project Title	**VaultMesh: A Versioned, Encrypted Cloud File Storage and Sharing System with Granular Access Control and Tamper-Evident Audit Trails**		
Date of Submission	29/09/2026		
Under taken at	RV College of Engineering		

Introduction:

Most file-sharing solutions in use today rely on a single cloud provider and a single shared server-side encryption key, creating three compounded risks: provider outages make all data unreachable, a breach of the provider's key store exposes every file simultaneously, and there is no durable record of who accessed or modified a file. Audit requirements under India's Digital Personal Data Protection (DPDP) Act and enterprise data-governance policies demand that sensitive files be protected with independent per-file keys, that every access event be logged in a tamper-evident manner, and that historical versions be preserved so that data loss and unauthorised modifications can be detected and reversed.

This project designs, implements, and evaluates VaultMesh: a cloud-hosted file storage and sharing platform in which every file is encrypted with an independent AES-256-GCM data key wrapped by a per-user master key derived from a passphrase using Argon2id, so that a compromise of the storage back-end does not expose plaintext. Files are versioned so that every upload produces an immutable snapshot rather than overwriting the previous one, enabling rollback to any prior state. A role-based access control layer enforces owner, editor, and viewer permissions at the file level. Every access, upload, permission change, and deletion is recorded in an append-only audit log whose integrity is maintained with cryptographic hash chaining, making retrospective tampering detectable. The result is a self-contained platform that demonstrates how object storage, managed databases, identity management, and cryptographic primitives compose into a trustworthy cloud application.

Objectives:

- **Secure per-file encryption:** implement client-side envelope encryption with a per-file AES-256-GCM data key (DEK) wrapped by a per-user key encryption key (KEK) derived from a passphrase using Argon2id, so the storage back-end never holds plaintext or bare DEKs.
- **Versioning and rollback:** ensure every file upload creates an immutable version record rather than overwriting previous content, and provide a restore operation that makes any historical version the current one.
- **Role-based access control:** implement an owner-editor-viewer permission model enforced server-side, with share and revoke operations that take effect immediately and are reflected in the audit log.
- **Tamper-evident audit trail:** record every access, mutation, and permission event in an append-only log where each entry carries a SHA-256 hash of the previous entry, making retrospective deletion or modification detectable.
- **Responsive web interface:** provide upload, download, version browsing, file sharing, permission management, and audit log inspection through a single-page application.
- **Evaluation:** measure encryption and decryption throughput, API response latency, version retrieval time, and correctness of access control enforcement under concurrent operations.

Methodology:

The system is built as a REST API service (Python/FastAPI) backed by AWS S3 (or MinIO for local development) for object storage and PostgreSQL for metadata, versioning, permissions, and audit records. It is containerised with Docker Compose. Development proceeds in the following steps:

1. **Key setup.** On registration, a KEK is derived from the user's passphrase using Argon2id with a per-user salt. The KEK is never stored; only the salt and the wrapped DEK are persisted.

2. **Upload and encryption.** On file upload, a fresh 256-bit DEK is generated. The file is encrypted with AES-256-GCM using a random nonce, and the DEK is wrapped with the user's KEK. The ciphertext is stored in S3 under a randomised object key; the wrapped DEK, nonce, object key, file name, and owner are written to PostgreSQL.
3. **Versioning.** Each upload creates a new version row linked to the logical file identifier. No existing row is modified. A version can be marked current; restoring a prior version inserts a new row pointing to the original ciphertext object, preserving the full history.
4. **Download and decryption.** The API checks the caller's permission, retrieves the wrapped DEK and nonce, re-derives the KEK from the supplied passphrase, unwraps the DEK, fetches the ciphertext from S3, and decrypts it. The GCM authentication tag is verified before any plaintext is returned.
5. **Access control.** A sharing table maps (file, user, role) triples. Every read and write operation queries this table server-side. Share and revoke operations are atomic and immediately reflected; role downgrades revoke write access on the next request.
6. **Audit logging.** Every authenticated operation appends a row to an audit table containing the actor, action, target, timestamp, and a SHA-256 hash over the previous row's hash concatenated with the current event, forming a hash chain. The chain can be verified by any party with read access to the log.
7. **Frontend.** A React (Vite) single-page application communicates with the API over HTTPS, providing file management, version history, sharing controls, and an audit log viewer with chain-verification status.
8. **Evaluation.** Locust load tests measure upload and download throughput and API latency. pytest and Hypothesis verify correctness of encryption, versioning, and access control under concurrent and adversarial inputs. Results are presented as benchmark graphs with a findings summary.

Data set used:

- No external or public dataset is used.
- A synthetic workload of files of varying sizes (1 KB to 200 MB) and MIME types is generated and uploaded through the API.
- Scripted adversarial scenarios: concurrent uploads of the same file, permission revocation mid-download, direct S3 object tampering to verify audit-chain detection, and rollback to versions following simulated corruption.
- The same scenarios are replayed across configurations (with and without hash-chain verification; versioned vs. overwrite mode) for comparison.

Software Requirements:

- API and gateway: Python, FastAPI, asyncio
- Cryptography: cryptography library (AES-256-GCM), argon2-cffi (Argon2id key derivation)
- Object storage: AWS S3 / MinIO (self-hosted); aioboto3 for async S3-compatible access
- Metadata, versioning, permissions, audit: PostgreSQL with SQLAlchemy (async)
- Frontend: React (Vite) with Recharts for version timelines and audit log visualisation
- Background tasks: APScheduler for periodic audit-chain integrity verification
- Testing and benchmarking: Locust, pytest, Hypothesis
- Containerisation and orchestration: Docker, Docker Compose
- Version control: Git/GitHub

Contribution to the field:

- Client-side encrypted storage and erasure-coded multi-cloud systems (RACS, DepSky, Tahoe-LAFS) demonstrate that keeping key material off the server is both feasible and desirable, but most implementations target research clusters rather than commodity cloud services.
- VaultMesh provides a lightweight, open, working implementation of per-file client-side encryption over standard S3-compatible object storage, integrated with a production-grade versioning and RBAC layer, making the approach accessible for everyday cloud deployment.
- The tamper-evident hash-chained audit log adds a data-integrity dimension absent from most storage-focused projects, and its verification algorithm can be extracted as an independent component for use in other append-only log systems.
- The project offers practical exposure to the principal concerns of cloud application development: scalable object storage, identity and access management, server-side security policy enforcement, and cloud-native observability.

Staff In charge:
Dr Nagaraja G.S
Professor and Associate Dean, RVCE
Bengaluru-560059